MySQL Cheat Sheet

MySQL 26.10 Innovation (LTS 9.7 / 8.4) · AI-generated summary — verify against the MySQL manual · see the MySQL pages for details

CLIENT — CONNECT

Connect

```
mysql -h 127.0.0.1 -P 3306 -u root -p shop
mysql -uroot -psecret shop -e "SELECT 1"
mysql ... --commands shop < script.sql
mysql ... --default-character-set=utf8mb4
```

Connection strings

```
mysql://user:password@host:3306/shop
jdbc:mysql://host:3306/shop?sslMode=REQUIRED
mysqlsh --sql --uri root:secret@host/shop
```

Client commands

```
\s status     \u shop   \G vertical
\. file.sql   source file.sql   help SELECT
SHOW TABLES;  DESCRIBE customers;
SHOW CREATE TABLE customers\G
```

Note

The client may default to latin1: pass --default-character-set=utf8mb4 for non-ASCII data.

TABLES — DEFINE

Create

```
CREATE TABLE IF NOT EXISTS notes (
  id   int AUTO_INCREMENT PRIMARY KEY,
  upd  timestamp DEFAULT CURRENT_TIMESTAMP
       ON UPDATE CURRENT_TIMESTAMP,
  kind enum('a','b') NOT NULL DEFAULT 'a',
  label varchar(10)
       AS (CONCAT('n-', kind)) STORED);
```

Alter, copy, drop

```
ALTER TABLE notes ADD COLUMN flag
  tinyint(1) NOT NULL DEFAULT 0,
  ALGORITHM=INSTANT;
CREATE TABLE notes_copy LIKE notes;
DROP TABLE IF EXISTS notes_old, notes_copy;
```

Data types

```
tinyint smallint mediumint int bigint
tinyint(1)    usual boolean
decimal(p,s)  money, never float/double
varchar(n) text enum(...) blob json
date datetime timestamp (UTC, 1970-2038)
```

DATA — MODIFY

Upsert

```
INSERT INTO customers (...) VALUES (...)
  AS new ON DUPLICATE KEY UPDATE
  name = new.name;
INSERT IGNORE INTO customers VALUES (...);
REPLACE INTO notes (id, body)
  VALUES (1, 'replaced');  -- delete+insert
```

Update, delete

```
UPDATE orders o JOIN customers c
  ON c.id = o.customer_id
  SET o.status='open' WHERE c.tier='free';
DELETE FROM customers WHERE id='c-9'
  ORDER BY id LIMIT 1;
SELECT LAST_INSERT_ID(), ROW_COUNT();
```

Query idioms

```
WITH t AS (SELECT customer_id, COUNT(*) n
  FROM orders GROUP BY customer_id)
SELECT * FROM t WHERE n > 1;
ROW_NUMBER() OVER (PARTITION BY customer_id
  ORDER BY order_date DESC)
GROUP_CONCAT(name ORDER BY name
  SEPARATOR ', ')
```

INDEXES — CREATE

Secondary, unique, full-text

```
CREATE INDEX idx
  ON orders (customer_id, order_date);
CREATE UNIQUE INDEX uq ON notes (id, kind);
CREATE INDEX idx ON t (col(20)); -- prefix
CREATE FULLTEXT INDEX ft ON notes (body);
MATCH(body) AGAINST ('mysql')
```

Functional, multi-valued (JSON)

```
CREATE INDEX idx ON items
  ((CAST(attrs->>'$.watts' AS UNSIGNED)));
CREATE INDEX idx ON items
  ((CAST(tags->'$' AS CHAR(20) ARRAY)));
```

Invisible

```
ALTER TABLE t ALTER INDEX idx INVISIBLE;
ALTER TABLE t ALTER INDEX idx VISIBLE;
```

Note

PRIMARY KEY is the clustered index. Composite indexes serve the leftmost prefix only.

PLANS — EXPLAIN

Forms

```
EXPLAIN SELECT * FROM orders
  WHERE customer_id='c-1';   -- TREE here
EXPLAIN FORMAT=TRADITIONAL SELECT ...;
EXPLAIN FORMAT=JSON SELECT ...;
EXPLAIN ANALYZE SELECT ...; -- runs it
SET explain_format = TRADITIONAL;
```

TREE output

```
-> Index lookup on orders
   using fk_orders_customer
   (customer_id = 'c-1')  (cost=0.8 rows=3)
```

Read the table form

type: ALL is a full scan; ref, range, const use an index. Also key, rows and Extra: Using filesort, Using temporary, Using index (covering).

TRANSACTIONS — LOCK

Isolation levels

```
READ UNCOMMITTED  dirty reads possible
READ COMMITTED    latest committed data
REPEATABLE READ   default, next-key locks
SERIALIZABLE      reads take shared locks
```

Set level

```
SELECT @@transaction_isolation;
SET SESSION TRANSACTION ISOLATION LEVEL
  READ COMMITTED;
```

Locking reads

```
SELECT ... FOR UPDATE;   -- exclusive lock
SELECT ... FOR SHARE;    -- shared lock
SELECT ... FOR UPDATE NOWAIT;
SELECT * FROM orders WHERE status='open'
  ORDER BY id LIMIT 1
  FOR UPDATE SKIP LOCKED;  -- job queue
```

Savepoints

```
SAVEPOINT s1;  ROLLBACK TO SAVEPOINT s1;
```

JSON — QUERY

Read and test

```
SELECT attrs->>'$.color'      -- oak
     , JSON_EXTRACT(attrs,'$.color')
  FROM products WHERE sku='A-1';
WHERE 'sale' MEMBER OF (tags)
SELECT JSON_ARRAYAGG(sku) FROM products;
```

Functions

```
JSON_VALUE(doc,path RETURNING type)
JSON_SET  JSON_INSERT  JSON_REPLACE
JSON_REMOVE  JSON_MERGE_PATCH
JSON_CONTAINS  JSON_OVERLAPS  JSON_VALID
JSON_OBJECT  JSON_ARRAY  JSON_OBJECTAGG
JSON_TABLE(doc,'$[*]' COLUMNS (...))
```

Paths

```
$  $.key  $[0]  $[last]  $[*]  $**.key
```

SERVER — ADMINISTER

Sessions, variables

```
SHOW FULL PROCESSLIST;  KILL QUERY 26;
SHOW GLOBAL STATUS LIKE 'Threads%';
SET GLOBAL max_connections = 300;
SET PERSIST max_connections = 300;
```

sys, performance_schema

```
SELECT * FROM sys.innodb_lock_waits;
performance_schema.data_locks
sys.statements_with_full_table_scans
sys.schema_unused_indexes
SHOW ENGINE INNODB STATUS\G
```

Users, roles

```
CREATE USER 'app'@'%' IDENTIFIED BY '...';
CREATE ROLE 'shop_read';
GRANT SELECT ON shop.* TO 'shop_read';
GRANT 'shop_read' TO 'app'@'%';
```

Note

SET GLOBAL lasts until restart; SET PERSIST survives it (mysqld-auto.cnf). Defaults seen: buffer pool 128 MiB, max_connections 151.

BACKUP — REPLICATE

Dump and restore

```
mysqldump -uroot -psecret \
  --single-transaction --routines \
  --triggers --events --source-data=2 \
  --databases shop > shop-full.sql
mysql -uroot -psecret < shop-full.sql
```

Point-in-time replay

```
mysqlbinlog --start-position=1795 \
  --stop-position=2781 binlog.000004 \
  | mysql -uroot -psecret
```

Replication

```
SHOW BINARY LOG STATUS;  -- on a source
SHOW REPLICA STATUS\G    -- on a replica
CHANGE REPLICATION SOURCE TO
  SOURCE_HOST='my-b', SOURCE_USER='repl',
  SOURCE_AUTO_POSITION=1, ...;
START REPLICA;  STOP REPLICA;
```

Note

On a replica check Replica_IO_Running, Replica_SQL_Running and Seconds_Behind_Source.